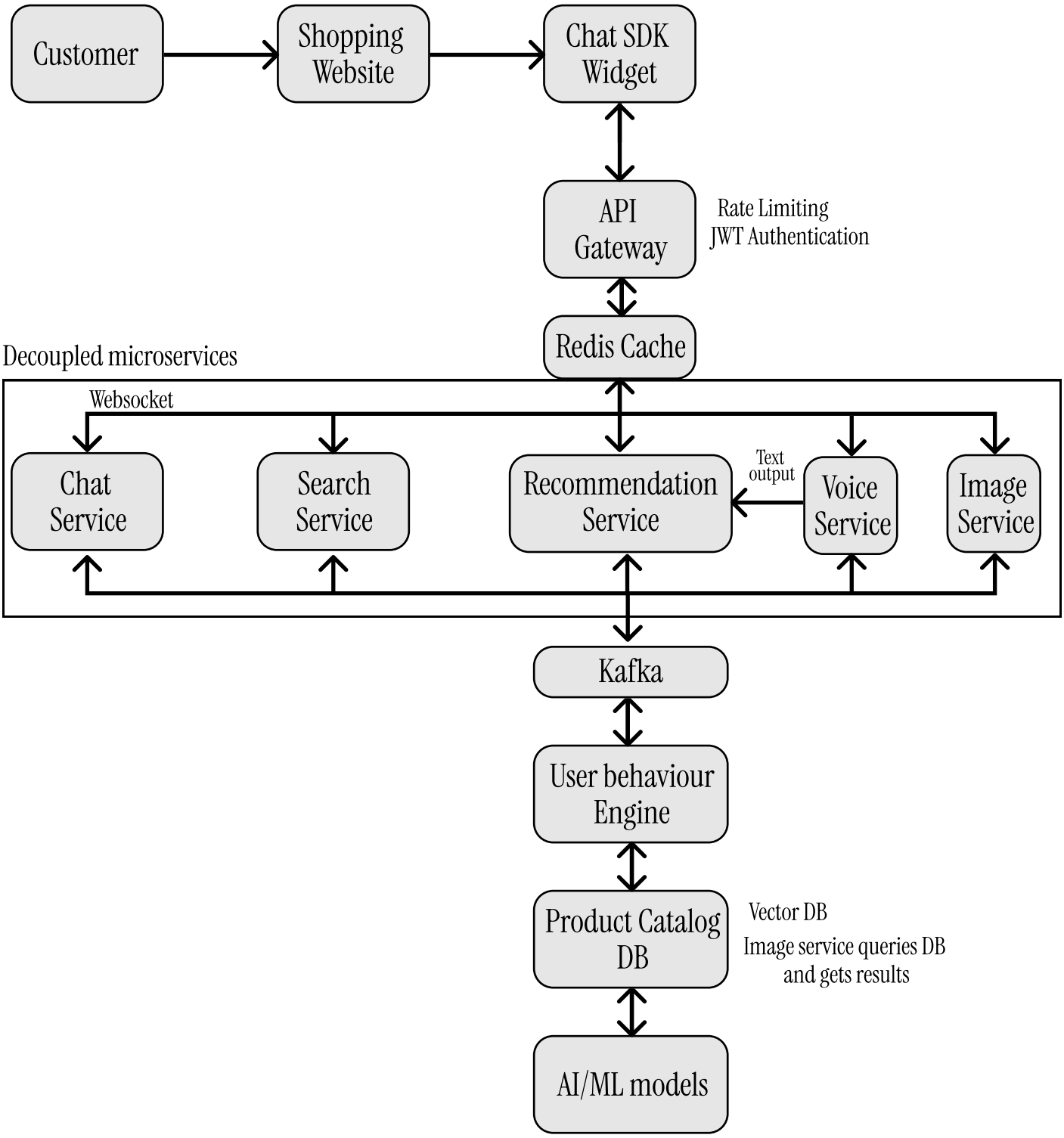
Date: 17/6/26

Chat SDK Design

High-Level Architecture:



Components:

Chat SDK:

- Tech → React, Typescript, Web socket
- A lightweight framework, Typescript for type safety of consumers and websockets for fast bi-directional realtime communication.

Date: 17/6/26

API Gateway:

- Tech → Spring Boot, Node.js, Kong Gateway
- Node.js for low latency, this also powers the web socket based chat service
- Spring Boot is robust and can effectively handle our Recommendation and AI engines.
- Kong gateway serves as the entry point to all the requests, providing security, rate limiting and effective routing.

Chat Service:

- It manages all the User messages, sessions and conversation history, through real-time web sockets.
- Here we use Redis to store active sessions and PostgreSQL to store persistent chats.

Search Service:

- We use Elasticsearch here because of its high performance in vector searches and its easy AI integration
- It then returns matching products and a relevance score.

Recommendation Service:

- It gets the data such as user image,voice, texts and processes it to fetch data from the vector database.
- It works with the historical data, click-stream data and current queries to get the ideal recommendation for the user.

Voice Service:

- User audio is captured and streamed to the backend.
- The server utilizes a multilingual ASR (Automatic Speech Recognition) model such as AI4Bharat IndicConformers(best for Indian languages) to transcribe audio into text before the recommendation engine processes the request.

Image Service:

- Takes user input images and compresses them to binary and send them to the server to be converted to vector embedding, we can then use KNN search on the product catalog to get similar results.

Storage:

- PostgreSQL → Stores Users, Orders and Chats
- Redis → Stores active sessions, frequently searched products
- Elasticsearch → Stores product catalogs
- Vector Database → Stores image and Product embedding

Scalability:
- Microservices → Chat, Recommendation, Search, Voice and Image
- Kafka → Events: Product Viewed, Product Clicked, Product Purchased , Chat Started. Consumers: Analytics, Recommendation, User profile services

Conclusion:

I would like to conclude this assignment by saying that this AI-powered Shopping Chat SDK makes product discovery faster and more personalized by allowing users to search through text, images, or voice in multiple languages. By combining user behavior data, recommendation algorithms, and modern search technologies, the system can provide relevant product suggestions while remaining scalable and easy to integrate with existing e-commerce platforms.

Relevant Past Experience:

While I have not built a complete Chat SDK, I have built several components that are closely related to the proposed architecture.

- Self-Optimizing SaaS Analytics Platform: This project gave me experience in designing scalable backend systems using a microservices architecture. I used Kafka for real-time event processing, PostgreSQL for data storage, and FastAPI-based services for machine learning inference. The project involved collecting user activity data, processing events in real time, and generating predictive insights. These concepts are similar to the recommendation engine and user behavior tracking components proposed in the shopping chat SDK.
- AI Healthcare Application: In this project, I developed a full-stack web application with an AI-powered chatbot and integrated frontend components with backend APIs. This experience helped me understand conversational interfaces, API integrations, and user-centric design principles that are directly applicable to building a chat-based shopping assistant.
- Real-Time Sports Broadcast Engine: I built a real-time application using Node.js and WebSockets capable of handling high-frequency events with low latency. This experience is relevant to the chat service layer of the SDK, where real-time communication and responsive user interactions are essential.

Github Repositories:

https://github.com/Devs7026/self-optimizing-saas

https://github.com/Devs7026/HealthCare

https://github.com/Devs7026/Sports_Broadcast_Engine